

# Git & GitHub for Absolute Beginners

---

This guide is written for someone who has **never used Git or GitHub before**. Follow it top to bottom, typing every command yourself — don't copy-paste blindly, since seeing what each step actually does is the point.

By the end you'll know: what Git and GitHub actually are, how to save your work's history, how to put a project on GitHub, how to share access with someone, how to copy someone else's project, and how to work on a feature safely and combine it back in.

---

## 0. What you need before starting

### 1. Install Git

- Windows/Mac/Linux installers: <https://git-scm.com/downloads>
- Check it worked: open a terminal and run

```
git --version
```

You should see something like `git version 2.4x.x`.

### 2. Create a free GitHub account at <https://github.com> if you don't have one.

### 3. (Optional but recommended) Install GitHub CLI — a tool that lets you do GitHub things (like creating a repo) from the terminal instead of clicking around a website.

- <https://cli.github.com>
- Check it worked:

```
gh --version
```

- Sign in once:

```
gh auth login
```

Follow the prompts — choose GitHub.com, HTTPS, and log in via the browser when asked.

### 4. Tell Git who you are (it needs this to label every change with your name):

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Use the same email as your GitHub account. Confirm it saved correctly:

```
git config --list
```

---

## 1. What's the difference between Git and GitHub?

This trips up almost everyone at first, so read this twice before touching the terminal.

- **Git** is a program that runs on *your computer*. It keeps a history of every change you make to your files — like an incredibly detailed "undo" log that never forgets anything, and lets you jump back to any earlier point.
- **GitHub** is a *website* that stores copies of Git projects online, so you can back them up, share them, and work on them with other people.

**Analogy:** Git is like a word processor's version history (it works even with no internet). GitHub is like Google Drive — a place on the internet where that document lives so others can see it, copy it, or edit it too.

You will use Git *constantly* (every few minutes while working). You'll use GitHub mainly when you want to save your work online, or work with others.

---

## 2. Creating and managing code with Git

Let's turn an ordinary folder into a project Git watches.

```
mkdir my-first-project
cd my-first-project
git init
```

`git init` tells Git "start tracking this folder." You'll now see a hidden `.git` folder inside — that's where all the history lives. You never need to open it.

Create a file:

```
echo "print('hello world')" > app.py
```

Now the three commands you'll use more than any others in Git:

```
git status
```

This tells you what's changed. Right now it'll say `app.py` is **untracked** — Git sees the file but isn't watching it yet.

```
git add app.py
```

This "stages" the file — it tells Git "include this file in my next save." Run `git status` again and you'll see it listed as staged, ready to be committed.

```
git commit -m "Add app.py with a hello world script"
```

This is the actual **save point** — a permanent snapshot in your project's history, with a short message describing what changed. The `-m` flag lets you write the message inline.

Check your history:

```
git log --oneline
```

**The habit to build:** every time you finish a meaningful chunk of work, repeat `status` → `add` → `commit`. Small, frequent commits with clear messages are far more useful later than one giant commit at the end.

---

### 3. Putting your project on GitHub (and adding a collaborator)

Right now your project only exists on your computer. If your laptop dies, it's gone. Let's back it up to GitHub.

**Create the GitHub repository and push your code, in one step,** using GitHub CLI:

```
gh repo create my-first-project --public --source=. --remote=origin --push
```

What this did:

- Created a new repository called `my-first-project` under your GitHub account
- Connected your local folder to it (called `origin`)
- Uploaded ( `push` ed) your commit history to it

Go to `github.com/<your-username>/my-first-project` in a browser — your file should be there.

(No GitHub CLI? Do it manually: on [github.com](https://github.com) click **New repository**, don't initialize it with a README, then copy the commands GitHub shows you under "...or push an existing repository from the command line".)

**Adding a collaborator** — someone you want to give direct edit access to:

1. Go to your repo on GitHub
2. **Settings** → **Collaborators** → **Add people**
3. Type their GitHub username, send the invite
4. They'll get an email/notification to accept

Once accepted, they can clone your repo and push changes directly, the same way you can.

---

## 4. Forking — copying someone else's project

A **fork** is what you use when you want to work on a project you *don't* have collaborator access to — for example, an open-source project, or a classmate's repo you're not added to.

1. Find a repo on GitHub (any public one works for practice).
2. Click the **Fork** button, top right of the page.
3. GitHub creates a full copy of it under *your own* account.
4. Clone your fork to your computer:

```
git clone https://github.com/<your-username>/<repo-name>.git
cd <repo-name>
```

Now you can make changes freely — you're editing *your copy*, not the original. If you later want to offer your changes back to the original project, you'd open a **Pull Request** from your fork (more on Pull Requests below).

**The difference to remember:**

- **Collaborator access** → you push directly to the original repo.
- **Fork** → you get your own copy first, and propose changes back later.

### Cloning vs. forking — these are not the same thing

It's easy to blur these together since both end with a copy of the project on your computer, but they happen in different places and solve different problems:

- **Cloning** ( `git clone <url>` ) happens on *your computer*, via Git. It just downloads whatever's already on GitHub — it doesn't create anything new on GitHub itself, and it works the exact same way whether or not you have write access.
- **Forking** happens *on GitHub*, by clicking the Fork button. It creates a brand-new repo under your account. You'd typically clone that fork afterward to actually work on it locally.

**So what actually changes when you have collaborator access vs. not?** Not the cloning step — that's identical either way. What changes is whether `git push` succeeds afterward:

|                                       | With collaborator access | Without collaborator access                                               |
|---------------------------------------|--------------------------|---------------------------------------------------------------------------|
| <code>git clone</code>                | Works, same as always    | Works, same as always (if the repo is public)                             |
| <code>git push</code> to the original | Succeeds                 | Rejected — GitHub checks your permissions                                 |
| What to do instead                    | Nothing — push directly  | Fork first, clone <i>your fork</i> , push there, then open a Pull Request |

**One line to remember:** `git clone` never checks permissions — it just downloads. Permissions only get checked the moment you try to `git push`. Forking exists purely as the workaround for when that push would otherwise fail.

## 5. Push and pull — the everyday sync loop

Once your project is on GitHub, these two commands are how you keep your computer and GitHub in sync.

**Push** = send your local commits up to GitHub.

```
echo "print('a second line')" >> app.py
git add app.py
git commit -m "Add a second print statement"
git push origin main
```

Refresh the repo page on GitHub — your new commit is there.

**Pull** = bring down changes that exist on GitHub but not yet on your computer (maybe someone else pushed something, or you edited the file directly on GitHub's website).

```
git pull origin main
```

Try it: edit a file directly in the GitHub website (click the file → pencil icon → edit → commit), then run `git pull origin main` on your computer and watch the change appear locally.

**Rule of thumb:** always `git pull` *before* you start working, so you're not about to edit an outdated copy of the file. Always `git push` once you're done, so your work isn't stuck only on your machine.

### What `git pull` is actually doing

`git pull` feels like one action, but it's really two steps back to back:

1. **Fetch** — Git contacts GitHub and downloads any commits that exist there but not on your machine yet. Nothing in your files changes yet; Git just now *knows* about the new commits.

2. **Merge** — Git merges those newly-downloaded commits into your current branch, the same kind of merge you'll use with branches in Section 6.

So `git pull origin main` is shorthand for:

```
git fetch origin main
git merge origin/main
```

Most of the time this merge is trivial and you won't even notice it happening. But if a teammate changed the exact same lines you have locally, you'll get a **merge conflict** — same mechanic as branch conflicts in Section 6, resolved the same way.

---

## 6. Branching and merging

So far everything has happened on one line of history (called `main`). A **branch** lets you work on something new without touching `main` until you're ready.

**Create and switch to a branch:**

```
git branch feature-goodbye
git checkout feature-goodbye
```

(or in one step: `git checkout -b feature-goodbye`)

Make a change on this branch:

```
echo "print('goodbye world')" >> app.py
git add app.py
git commit -m "Add goodbye print"
```

**Want to see this branch on GitHub before merging it?** Push it up by name, same as you would `main`:

```
git push origin feature-goodbye
```

Now go to your repo on GitHub and click the branch dropdown (it defaults to showing `main`) — `feature-goodbye` will be listed there. Selecting it shows the repo exactly as it looks on that branch, including your goodbye line, while `main` on GitHub stays untouched. This is the same idea as Section 6 below, just visible in the browser instead of only on your machine.

Switch back to `main` and notice your change isn't there — it's safely isolated on the branch:

```
git checkout main
cat app.py
```

`cat app.py` just prints the file's contents to the screen so you can see them — it's not a Git command, it's a general terminal command (short for "concatenate"). Here it's a quick way to check that the goodbye line is *not* present on `main`, since that change only exists on `feature-goodbye` so far.

**Merge** brings the branch's changes into `main`:

```
git merge feature-goodbye
cat app.py    # now the goodbye line is here too
```

**What if both branches changed the same line?** Git can't guess which version you want, so it stops and asks you to decide — this is called a **merge conflict**. It looks like this inside the file:

```
<<<<<<< HEAD
this is main's version
=====
this is the branch's version
>>>>>>> feature-goodbye
```

To resolve it: edit the file to keep the version you want (deleting the `<<<<<<<`, `=====`, `>>>>>>>` markers), then:

```
git add app.py
git commit -m "Resolve conflict"
```

**Don't panic when you see a conflict** — it isn't an error, it's Git correctly refusing to guess. Read both versions, decide, and move on.

## Seeing all your branches and checking their status

Once you have more than one branch, you'll want to check what exists and where you stand.

**List your local branches** (a `*` marks the one you're currently on):

```
git branch
```

**List local *and* remote branches** (ones that exist on GitHub but you may not have checked out locally):

```
git branch -a
```

**List only the remote branches:**

```
git branch -r
```

**See more detail per branch** — last commit, and how far ahead/behind it is from GitHub:

```
git branch -vv
```

**Check which branch you're on right now, plus any uncommitted changes** — this is the command you'll run constantly:

```
git status
```

The first line always tells you `On branch <name>`.

**See the branch/merge history visually:**

```
git log --oneline --graph --all
```

This draws your commits and branches as ASCII art, which makes it much easier to see how branches diverged and merged than `git branch` alone.

**Check which branches are already merged in** (useful for cleanup — these are generally safe to delete):

```
git branch --merged  
git branch --no-merged
```

---

## 7. Putting it all together: a Pull Request

A **Pull Request (PR)** is how you propose merging a branch (or a fork's changes) into the main project on GitHub, so someone can review it first.

### Option 1 — GitHub CLI (fastest)

Make sure your branch is pushed:

```
git push origin feature-goodbye
```

Then create the PR:

```
gh pr create --fill
```

`--fill` auto-fills the title and description from your branch's commit messages. It'll ask which branch to merge into (defaults to `main`) and print a link to the new PR.

To write your own title and description instead of auto-filling:



```
gh pr create --title "Add goodbye print" --body "Adds a goodbye message to app.py"
```

## Option 2 — GitHub website

```
git push origin feature-goodbye
```

Then on GitHub:

1. You'll usually see a yellow banner: **"feature-goodbye had recent pushes"** with a **Compare & pull request** button — click it. *(No banner? Go to the **Pull requests** tab → **New pull request** → pick your branch from the dropdown.)*
2. Check the **base** (what you're merging into, usually `main`) and **compare** (your branch) are correct.
3. Add a title and description explaining what changed and why.
4. Click **Create pull request**.

On GitHub, the PR page shows exactly what changed. A reviewer can comment, request changes, or approve — then click **Merge**. This is the same `merge` idea from Section 6, just done through GitHub's interface with a review step in between.

**Note:** you can only open a PR if your branch has commits that differ from the target branch — if they're identical, GitHub won't let you create one.

## Who opens the PR, and who merges it?

Not fixed by Git or GitHub — it's team convention — but there's a strong common pattern:

- **Who opens (proposes) it:** almost always **the developer** who did the work. They finished a branch, pushed it, and are asking "please review and merge my changes." That's literally what "Pull Request" means.
- **Who merges it:** typically **whoever reviewed and approved it** — a senior developer, a peer (in small teams: "you review mine, I review yours"), or the repo maintainer. Many teams also let the original author click Merge themselves once someone else has approved.

A Scrum Master generally isn't part of this step — that role is about facilitating process (standups, removing blockers, keeping the sprint on track), not technical review. If they're also a developer wearing two hats, they could review/merge in that capacity, but not because of the Scrum Master title itself.

### Typical flow, end to end:

1. Developer finishes a branch → opens the PR
2. A reviewer (peer, senior dev, or lead) looks it over, comments, requests changes if needed
3. Once approved, someone clicks Merge — either the reviewer or the original developer, depending on team convention

Don't confuse this with `git pull` — they share a word, nothing else

|                        | <code>git pull</code>                                   | Pull Request                                          |
|------------------------|---------------------------------------------------------|-------------------------------------------------------|
| Where it happens       | Your terminal                                           | GitHub website                                        |
| What it does           | Downloads + merges commits from GitHub into your branch | Proposes merging one branch into another, with review |
| Approval needed?       | No — happens instantly                                  | Usually yes — a human clicks Merge                    |
| How often you'd use it | Many times a day, just to stay in sync                  | Once per feature or change you want reviewed          |

The naming connection: a Pull Request is really asking the maintainer "please **pull** my changes in" — from their point of view, merging your PR is like them pulling your commits into their branch. That's the actual link. As commands, though, `git pull` (Section 5) and a Pull Request (this section) do completely different jobs — don't mix them up.

## 8. CI/CD — automating your checks

By now you've got a full workflow: write code, commit, push, branch, merge, open PRs. **CI/CD** automates the checks you'd otherwise have to remember to run by hand before every push.

- **CI (Continuous Integration)**: every push or PR automatically runs your tests (and other checks like linting) so nobody has to remember to run them manually.
- **CD (Continuous Delivery/Deployment)**: once those checks pass, automatically package (or deploy) the result.

This lives on GitHub, in a folder called **GitHub Actions**. Let's set up a simple one on `my-first-project`.

### Step 1 — add something worth testing

So far `app.py` just prints text. Let's give it one real function and a test for it.

```
git checkout main
git pull origin main
```

Replace the contents of `app.py` :

```
def add(a, b):  
    return a + b  
  
if __name__ == "__main__":  
    print("hello world")
```

Create a tests folder and a test file:

```
mkdir tests
```

Create `tests/test_app.py`:

```
from app import add  
  
def test_add():  
    assert add(2, 3) == 5
```

Add a `requirements.txt` so GitHub knows what to install:

```
echo "pytest" > requirements.txt
```

Try it locally first — this is the exact check CI is about to automate for you:

```
pip install pytest  
pytest tests/ -v
```

You should see `1 passed`. Commit and push:

```
git add .  
git commit -m "Add add() function with a test"  
git push origin main
```

## Step 2 — write the workflow file

GitHub Actions workflows are `.yaml` files that live in a specific folder Git expects:

```
mkdir -p .github/workflows
```

Create `.github/workflows/ci.yaml`:

```

name: CI pipeline

on:
  push:
    branches: ['**']
  pull_request:
    branches: ['**']

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.13"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Run tests
        run: pytest tests/ -v

```

Quick translation of the pieces:

- **on:** — what triggers this. Here: any push, and any Pull Request, on any branch.
- **jobs: test:** — a job named **test**, running on a fresh Ubuntu virtual machine.
- **steps:** — the actual tasks. **uses:** runs a pre-built Action (reusable functionality someone else published, like "check out my repo" or "install Python"). **run:** executes a plain shell command, same as you'd type locally.

Commit and push it:

```

git add .github/workflows/ci.yml
git commit -m "Add CI workflow to run tests automatically"
git push origin main

```

### Step 3 — watch it run

On GitHub, go to your repo's **Actions** tab. You'll see the workflow running (or already finished) — click into it, then click the **test** job to see the live log of the exact same **pytest** output you saw locally, except GitHub ran it for you automatically the moment you pushed.

## Step 4 — see it catch a real problem

This is the actual point of CI — not the green checkmark, but the fast feedback when something's broken.

Edit `tests/test_app.py` so it deliberately fails:

```
from app import add

def test_add():
    assert add(2, 3) == 6    # wrong on purpose
```

```
git add tests/test_app.py
git commit -m "Intentionally break the test"
git push origin main
```

Go back to the **Actions** tab — the run shows a red **X** this time. Click into the log and read the actual pytest failure. This is CI doing its job: telling you something's wrong within seconds of pushing, instead of you finding out later (or a teammate finding out for you).

Fix it and push again:

```
git revert HEAD --no-edit
git push origin main
```

Watch it turn green again.

## Step 5 — CI gating a Pull Request

This ties Section 7 and Section 8 together. Open a PR (as in Section 7) and look at the PR page — you'll see the same CI check running directly on it, before GitHub even lets you merge. That's the real payoff: CI isn't just "runs after I push," it's a gate that catches problems *before* bad code reaches `main`.

---

## Quick reference — commands used in this guide

| Command                                      | What it does                                       |
|----------------------------------------------|----------------------------------------------------|
| <code>git init</code>                        | Start tracking a folder                            |
| <code>git status</code>                      | Show what's changed                                |
| <code>git add &lt;file&gt;</code>            | Stage a change                                     |
| <code>git commit -m "..."</code>             | Save a snapshot                                    |
| <code>git log --oneline</code>               | View history                                       |
| <code>git push origin main</code>            | Send commits to GitHub                             |
| <code>git pull origin main</code>            | Bring GitHub's changes down                        |
| <code>git clone &lt;url&gt;</code>           | Copy a repo to your computer                       |
| <code>git branch &lt;name&gt;</code>         | Create a branch                                    |
| <code>git branch</code>                      | List local branches                                |
| <code>git branch -a</code>                   | List local + remote branches                       |
| <code>git branch -vv</code>                  | List branches with last commit + ahead/behind info |
| <code>git branch --merged</code>             | List branches already merged in                    |
| <code>git log --oneline --graph --all</code> | Visualize branch/merge history                     |
| <code>git checkout &lt;name&gt;</code>       | Switch to a branch                                 |
| <code>git checkout -b &lt;name&gt;</code>    | Create + switch in one step                        |
| <code>git merge &lt;name&gt;</code>          | Combine a branch into your current one             |
| <code>gh repo create</code>                  | Create a GitHub repo from the terminal             |
| <code>gh repo add-collaborator</code>        | Add someone to your repo                           |
| <code>gh pr create --fill</code>             | Open a Pull Request (CLI)                          |
| <code>.github/workflows/*.yaml</code>        | Where GitHub Actions (CI/CD) workflows live        |

**The mental model to keep:** Git saves history on your machine ( `add` → `commit` ). GitHub stores and shares that history online ( `push` / `pull` ). Branches let you experiment safely; merging (directly, or via a Pull Request) brings the work back together. CI/CD (GitHub Actions) automates the checks — like running your tests — every time you push or open a PR, so nobody has to remember to run them by hand.